# Meshtastic Position Data Flow

How positions get from devices in the field to the Aervyx live tracking map.

## Overview

There are **3 paths** a position can take from the field to the live map:

| Path | What sends the position | Internet needed on | Latency |
|---|---|---|---|
| **1 - Phone Direct** | Phone sends its own location over cellular | Pilot's phone | ~1s |
| **2 - BLE Relay** | Phone forwards mesh packets to Aervyx API | Pilot's phone | ~2-5s |
| **3 - MQTT** | Gateway node publishes to MQTT broker | WiFi/Ethernet gateway | ~5-15s |

```
flowchart TB
    subgraph field["In the Field"]
        direction LR

        subgraph pilotA["Pilot A -- no cell service"]
            A_DEV["Meshtastic\nDevice A"]
            A_PH["Phone A\nno internet"]
        end

        subgraph pilotB["Pilot B -- has cell service"]
            B_DEV["Meshtastic\nDevice B"]
            B_PH["Phone B\ncell + BLE"]
        end

        subgraph gw["Stationary WiFi Gateway"]
            GW_DEV["Meshtastic Device\nWiFi + MQTT enabled"]
        end

        subgraph wild["Public Mesh Node"]
            W_DEV["Stranger's device\nwith MQTT uplink"]
        end
    end

    A_DEV <--->|"LoRa radio\nhop_limit=3"| B_DEV
    A_DEV <--->|"LoRa"| GW_DEV
    A_DEV <--->|"LoRa"| W_DEV
    B_DEV <--->|"LoRa"| GW_DEV

    A_DEV <-->|"BLE"| A_PH
    B_DEV <-->|"BLE"| B_PH

    B_PH -->|"PATH 1: Phone Direct\nHTTP POST source=app\n(phone's own location)"| API
    B_PH -->|"PATH 2: BLE Relay\nHTTP POST source=mesh_relay\nforwards ALL heard packets"|
API
```

```
    GW_DEV -->|"MQTT publish"| PUB
    W_DEV  -->|"MQTT publish"| PUB
    GW_DEV -.->|"MQTT publish\nif private configured"| PRIV

    subgraph brokers["MQTT Brokers"]
        PUB["mqtt.meshtastic.org\nPublic broker"]
        PRIV["Private broker\nyour-broker.example.com"]
    end

    subgraph aervyx["Aervyx Backend"]
        API["REST API\n/api/track/position"]
        MSUB["MQTT Subscriber"]
        API  --> RES["Resolve pilot_id\nauth token or\nmesh_device_id lookup"]
        MSUB --> RES
        RES  --> TASK["Resolve task_id\nfrom active task"]
        TASK --> DB[("TrackingPoint")]
        DB   --> SSE["SSE fan-out"]
    end

    PUB  -->|"PATH 3a: subscribe\nmsh/+/+/position"| MSUB
    PRIV -.->|"PATH 3b: subscribe"| MSUB

    SSE --> MAP["Live Tracking Map"]
```

---

## Path 1 -- Phone Direct (cellular)

The pilot's phone has cell service. The Aervyx mobile app sends the phone's own location directly to the backend over cellular data. No Meshtastic device is involved in this path -- it is pure phone-to-server.

| Step | What happens |
|---|---|
| Phone location fix | Android/iOS location services provide lat/lon/alt |
| HTTP POST | `POST /api/track/position` with `source=app` |
| Auth | Bearer token identifies the pilot |
| Store + broadcast | Position saved, SSE pushed to live map |

**When it works:** Pilot has cell service. Lowest latency, simplest path. No mesh radio needed.

---

## Path 2 -- BLE Relay (phone bridges mesh to API)

The phone is paired to a Meshtastic device via Bluetooth Low Energy. The device receives LoRa packets from the entire mesh network. The phone app reads those packets, parses the protobuf, and forwards every position to the backend.

| Step | What happens |
|---|---|
| LoRa broadcast | Any mesh device broadcasts its position |

| | |
|---|---|
| Relay / rebroadcast | Other mesh nodes repeat the packet (up to `hop_limit`) |
| BLE delivery | Phone reads `fromRadio` BLE characteristic |
| Protobuf parse | App extracts MeshPacket -> Position (lat, lon, alt, speed) |
| HTTP POST | `POST /api/track/position` with `source=mesh_relay`, `device_id=!XXXXXXXX` |
| pilot_id lookup | Backend matches device_id to `users.mesh_device_id` |
| task_id lookup | Backend finds pilot's active task from event registration |
| Store + broadcast | Position saved, SSE pushed to live map |

**Key detail:** This path forwards **all** mesh positions heard by the connected device, not just its own. If Pilot B's phone has cell service and Pilot A is beyond cell range but within LoRa range, Pilot A's position reaches Aervyx through Pilot B's phone.

---

## Path 3 -- MQTT (mesh to broker to Aervyx subscriber)

A Meshtastic device with internet access (WiFi or Ethernet) and MQTT enabled acts as a gateway. It publishes every mesh packet it receives to an MQTT broker. The Aervyx backend subscribes to that broker.

| Step | What happens |
|---|---|
| LoRa broadcast | Device broadcasts position via radio |
| Mesh relay | Other nodes rebroadcast (up to `hop_limit`) |
| Gateway receives | Any MQTT-enabled node with internet hears the packet |
| MQTT publish | Gateway publishes to broker topic `msh/US/2/e/position/...` |
| Aervyx subscribes | MQTT subscriber receives the ServiceEnvelope message |
| Protobuf parse | ServiceEnvelope -> MeshPacket -> Position |
| pilot_id lookup | `from_node` -> `!XXXXXXXX` -> `users.mesh_device_id` |
| task_id lookup | Active task resolution from event registration |
| Store + broadcast | Position saved, SSE pushed to live map |

---

## Public vs Private MQTT

### Public broker (mqtt.meshtastic.org)

```
Your devices --> mesh --> ANY node with MQTT uplink --> broker
                          (yours OR a stranger's)
```

- Broader coverage: random hikers, other pilots, and public gateways all relay your positions for free
- Zero infrastructure: Meshtastic runs the broker
- Your positions are visible to anyone subscribing to the public broker

- You share bandwidth with all Meshtastic traffic worldwide

**Private broker (self-hosted)**

```
Your devices --> mesh --> only YOUR configured nodes --> your broker
```

- Private: only you see the data
- Dedicated bandwidth, no shared noise
- No benefit from public nodes -- strangers cannot relay to your broker because they do not have your credentials
- You must run and maintain the broker infrastructure

---

## Scenario: Pilot A is out of cell range

```
Pilot A (no cell)          Pilot B (has cell)           Aervyx

     |                          |                         |
     | -- LoRa broadcast ---->  |                         |
     |    (position packet)     |                         |
     |                          |                         |
     |                    Phone receives                  |
     |                    packet via BLE                  |
     |                    from Device B                   |
     |                          |                         |
     |                          | -- HTTP POST -------->  |
     |                          |    source=mesh_relay     |
     |                          |    device_id=!A_ID       |
     |                          |                         |
     |                          |                   Resolve:
     |                          |                   !A_ID -> Pilot A
     |                          |                   Pilot A -> active task
     |                          |                         |
     |                          |                   Store + SSE broadcast
     |                          |                         |
     |                          |                   Pilot A appears
     |                          |                      on live map
```

Pilot A never had internet, but their position reached the map through the mesh network and Pilot B's phone acting as a relay.

---

## Device ID auto-registration

When a pilot connects to a Meshtastic device via BLE in the mobile app, the app automatically registers that device's node ID ( `!XXXXXXXX` ) against the pilot's account. No manual configuration is needed.

If the pilot switches to a different device, the new device ID overwrites the old one. This means the MQTT subscriber and BLE relay can always resolve incoming mesh positions to the correct pilot.

```
sequenceDiagram
    participant Phone as Mobile App
```

```
    participant Device as Meshtastic Device
    participant API as Aervyx API
    participant DB as Database

    Phone->>Device: BLE connect
    Device-->>Phone: Config dump (includes myNodeNum)
    Phone->>Phone: Format node ID: !XXXXXXXX
    Phone->>API: PUT /api/auth/mesh-device<br>{"mesh_device_id": "!abcd1234"}
    API->>DB: UPDATE users SET mesh_device_id = '!abcd1234'<br>WHERE id = current_user.id
    API-->>Phone: 200 OK
    Note over Phone,DB: Future mesh positions from !abcd1234<br>now resolve to this pilot
```

---

## Live tracking works for everyone

Positions are stored and broadcast regardless of whether the pilot is in an active competition task. The live tracking map has three viewing modes:

| Mode | What it shows | SSE endpoint |
|---|---|---|
| **Task** | Pilots in a specific competition task | `/api/track/live/{taskId}` |
| **Buddy group** | Selected pilot IDs | `/api/track/live/pilots?ids=...` |
| **All users** | Every active position (public) | `/api/public/live/all` |

A pilot flying free (no active task) still appears in "All users" and buddy group views. The task_id is optional -- it enables the task-scoped competition view but is not required for tracking to work.

---

## Requirements for each path

| Requirement | Path 1 (Phone Direct) | Path 2 (BLE Relay) | Path 3 (MQTT) |
|---|---|---|---|
| Meshtastic device needed | No | Yes (paired via BLE) | Yes (any node with MQTT) |
| Phone has cell service | Yes | Yes | No |
| MQTT gateway node exists | No | No | Yes |
| Device auto-registered to user | No | Yes (auto on connect) | Yes (auto on connect) |
| Pilot registered for active task | No | No | No |
| What provides internet | Phone cellular | Phone cellular | Gateway WiFi/Ethernet |